

Lab 04

Machine Learning

SNS EE/CE 252L/251L-T2

Name: M Shees Abbas

ID: ma11212

Task 1



Task 2

step 1

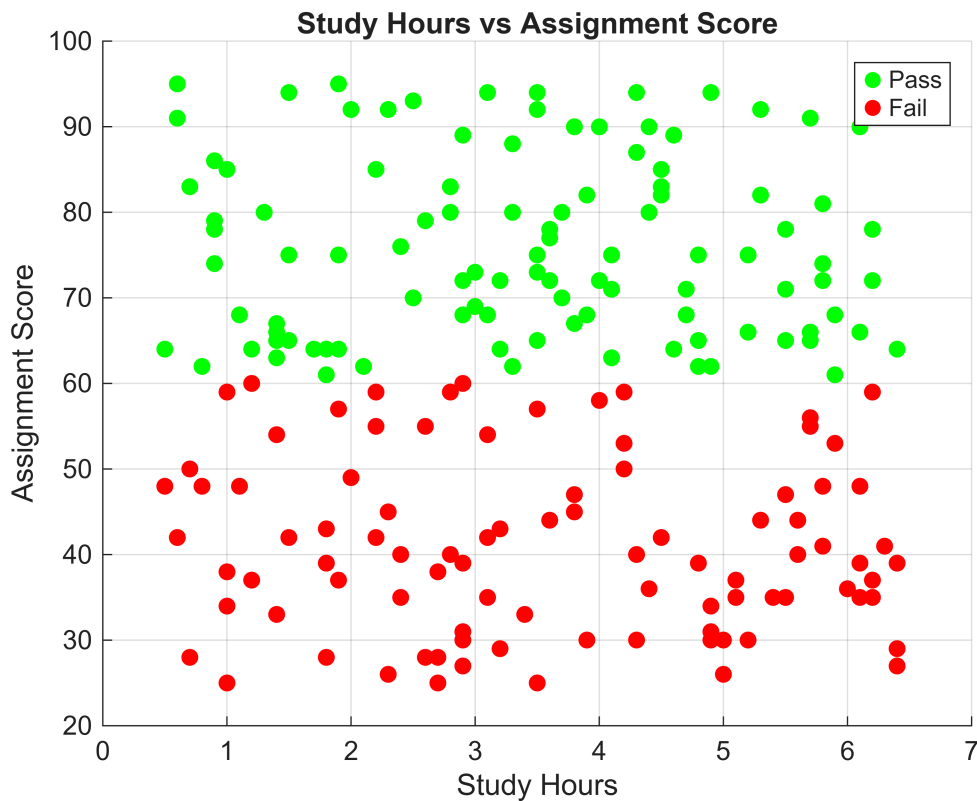
```
data = readtable("student_data.xlsx");  
disp(head(data, 5));
```

StudyHours	Attendance	AssignmentScore	Result
1.5	94	65	{'Pass'}
2.8	36	83	{'Pass'}
4.2	68	59	{'Fail'}
1.2	38	64	{'Pass'}
0.5	78	48	{'Fail'}

```

passrow = strcmp(data.Result, 'Pass');
failrow = strcmp(data.Result, 'Fail');
figure;
scatter(data.StudyHours(passrow), data.AssignmentScore(passrow), 40, 'g',
'filled');
hold on;
scatter(data.StudyHours(failrow), data.AssignmentScore(failrow), 40, 'r',
'filled');
hold off;
xlabel('Study Hours');
ylabel('Assignment Score');
title('Study Hours vs Assignment Score');
legend('Pass', 'Fail');
grid on;

```



we used the scatter function to plot the graph because we didnt want a continious graph because we are not plotting these against time, and both of the components are discrete

step 2

```
StudyHours = data.StudyHours;  
Attendance = data.Attendance;  
AssignmentScore = data.AssignmentScore;  
features = [StudyHours, Attendance, AssignmentScore];  
labels = data.Result;
```

step 3

```
featuresNorm = normalize(features, "zscore");
```

step 4

```
rng(42);  
numStudents = size(featuresNorm, 1);  
shuffledIndices = randperm(numStudents);  
trainCount = round(0.8 * numStudents);  
  
XTrain = featuresNorm(shuffledIndices(1:trainCount), :);  
yTrain = labels(shuffledIndices(1:trainCount));  
XTest = featuresNorm(shuffledIndices(trainCount+1:end), :);  
yTest = labels(shuffledIndices(trainCount+1:end));
```

step 5

```
logisticModel = fitcsvm(XTrain, yTrain);  
  
predLogistic = predict(logisticModel, XTest);  
accuracyLogistic = mean(strcmp(predLogistic, yTest)) * 100;  
fprintf('Logistic Regression Accuracy: %.1f%%\n', accuracyLogistic);
```

Logistic Regression Accuracy: 97.5%

```
confMatrixLogistic = confusionmat(yTest, predLogistic);  
disp('Confusion Matrix:');
```

Confusion Matrix:

```
disp(confMatrixLogistic);
```

```
19    0  
 1   20
```

step 6

```
treeModel = fitctree(XTrain, yTrain);  
predTree = predict(treeModel, XTest);  
accuracyTree = mean(strcmp(predTree, yTest)) * 100;
```

```
fprintf('Decision Tree Accuracy: %.1f%%\n', accuracyTree);
```

Decision Tree Accuracy: 100.0%

```
confMatrixTree = confusionmat(yTest, predTree);  
disp('Confusion Matrix:');
```

Confusion Matrix:

```
disp(confMatrixTree);
```

```
19    0  
0    21
```

```
forestModel = TreeBagger(30, XTrain, yTrain, 'Method', 'classification');  
predForest = predict(forestModel, XTest);  
predForest = strtrim(predForest);  
accuracyForest = mean(strcmp(predForest, yTest)) * 100;  
fprintf('Random Forest Accuracy: %.1f%%\n', accuracyForest);
```

Random Forest Accuracy: 100.0%

```
confMatrixForest = confusionmat(yTest, predForest);  
disp('Confusion Matrix:');
```

Confusion Matrix:

```
disp(confMatrixForest);
```

```
19    0  
0    21
```

```
svmModel = fitcsvm(XTrain, yTrain);  
predSVM = predict(svmModel, XTest);  
accuracySVM = mean(strcmp(predSVM, yTest)) * 100;  
fprintf('SVM Accuracy: %.1f%%\n', accuracySVM);
```

SVM Accuracy: 97.5%

```
confMatrixSVM = confusionmat(yTest, predSVM);  
disp('Confusion Matrix:');
```

Confusion Matrix:

```
disp(confMatrixSVM);
```

```
19    0  
1    20
```

step 7

```
[bestAccuracy, bestIndex] = max([accuracyLogistic, accuracyTree,  
accuracyForest, accuracySVM]);
```

```

modelNameNames = {'Logistic Regression', 'Decision Tree', 'Random Forest',
'SVM'};
fprintf('\nBest Model: %s with %.1f%% accuracy\n', modelNameNames{bestIndex},
bestAccuracy);

```

Best Model: Decision Tree with 100.0% accuracy

step 8

```

fprintf('\n=== TEST NEW STUDENT ===\n');

```

=== TEST NEW STUDENT ===

```

newStudent = [5, 85, 78];
newStudentNorm = (newStudent - mean(features)) ./ std(features);
if bestIndex == 1
    prediction = predict(logisticModel, newStudentNorm);
elseif bestIndex == 2
    prediction = predict(treeModel, newStudentNorm);
elseif bestIndex == 3
    prediction = predict(forestModel, newStudentNorm);
    prediction = strtrim(prediction);
else
    prediction = predict(svmModel, newStudentNorm);
end

fprintf('New Student: Hours=%d, Attendance=%d%, Score=%d\n', newStudent(1),
newStudent(2), newStudent(3));

```

New Student: Hours=5, Attendance=85%, Score=78

```

fprintf('Predicted Result: %s\n', prediction{1});

```

Predicted Result: Pass